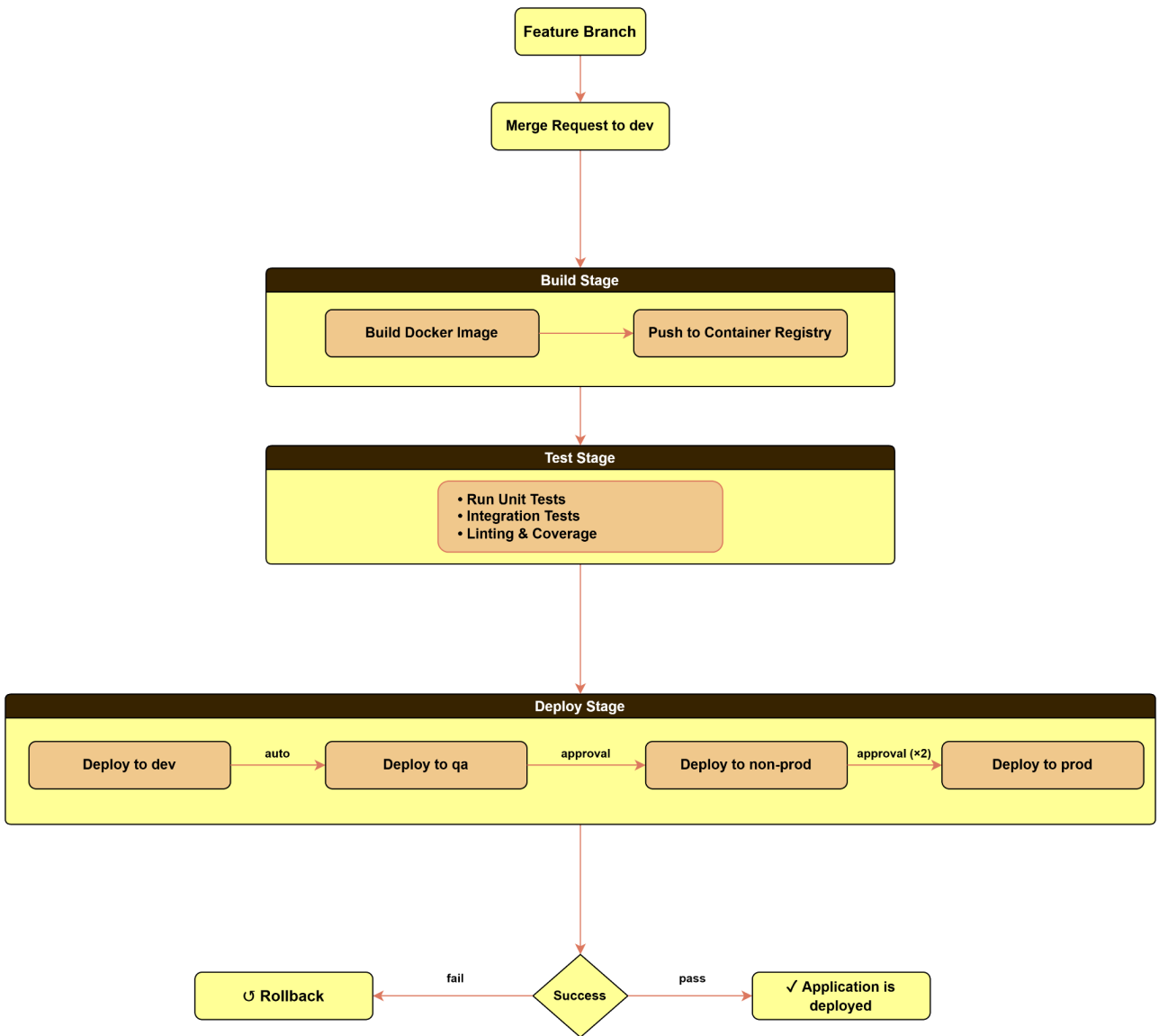


Berkeley Visitor Counter

CI/CD Pipeline Plan

Tool: GitLab CI/CD



Branching & Environment Promotion Strategy

Feature Branch → Dev

- Developer creates a feature branch
- Changes are implemented to the feature branch and a Merge Request is raised to the **dev** branch
- Upon merge, the CI/CD pipeline automatically triggers
- Application is built, tested, and deployed to the **Dev environment**

Dev → QA

- After successful deployment and validation in Dev, promotion to **QA** is initiated
- Promotion requires approval from authorized reviewers
- CI/CD pipeline deploys the same version to the **QA environment**
- QA performs functional and regression testing

QA → Non-Prod

- After QA sign-off, promotion to **Non-Prod** is initiated
- Promotion requires approval from authorized reviewers
- CI/CD pipeline deploys to the **Non-Production environment**
- Used for staging and release readiness validation

Non-Prod → Production

- After successful validation in Non-Prod, promotion to **Production** is initiated
- Requires controlled approvals from designated approvers
- CI/CD pipeline deploys to the **Production environment**
- Ensures only fully validated and approved versions are released

CI/CD Pipeline Stages

Stage 1: Build

Trigger: Merge to **dev** branch

Steps:

- Checkout source code
- Install dependencies
- Build Docker image
- Tag image using commit version
- Push image to Container Registry (like **Amazon ECR**)

Stage 2: Test

Trigger: After successful Building and pushing of Docker image

Steps:

- Run unit tests
- Run code quality and linting checks (using a tool like **SonarQube**)

Success Criteria: All tests pass successfully

Stage 3: Deploy

Trigger: After successful completion of the Test stage

Approvals Required:

- Pipeline will require approval before promotion to **QA**, **Non-Prod** and **Prod** environment

Steps:

- CI/CD pipeline deploys the application to the target **Amazon EKS** cluster
- Pipeline authenticates with the appropriate **EKS** environment
- Deployment configuration is updated with the new container image version
- Pipeline executes deployment using *kubectl* commands or Helm chart upgrade/install as part of the deploy stage
- Deployment rollout and pod status are monitored to ensure successful deployment
- Smoke tests are executed to verify application health and availability

- Team is notified of deployment success or failure via email, Slack, etc.

Rollback:

- If deployment health checks fail or rollout is unsuccessful, the pipeline automatically rolls back to the previous stable version

Success Criteria:

- Application is successfully deployed to the **EKS** cluster
- Pods are running and stable
- Deployment rollout completes successfully
- Smoke tests pass and application responds correctly